

Investigate the properties of FTA through empirical experimentation

1st Harsh Raj

Computer Science and Engineering
SRM Institute of Science and Technology
Delhi, India
raj.harsh1906@gmail.com

2nd Dilshad Shaik

Computer Science
GITAM University
Andhra Pradesh, India
sheikdilshad1@gmail.com

Abstract—Recently, the rapid development of artificial intelligence has led to the introduction of an array of different technologies. One such technology is fuzzy tiling activation. ReLU activation has been shown to work better for deep learning neural networks than other types of activation functions [1], but fuzzy tiling is a new type of activation function that has not been tried before. This work investigates the properties of FTA. Our work found that the use of tanh normalization is not dependent on bounds and can enhance the performance of FTA without any need for further tuning in the given environment. As a future direction, exploring the robustness of tanh-normalized DQN-FTA across multiple environments would be an intriguing prospect [2].

I. INTRODUCTION

Neural networks (NN) have been shown to perform well on several challenging problems [3]–[6]. Their efficacy stems from a knack for learning valid summaries of the input information through which appropriate representations are constructed. A representation of input is defined as the values of all the connection points in an artificial neural network, barring those present in the final layer of the NN when the input is passed into the NN [7]. The values of all the neurons are commonly called *features*. Unfortunately, when the NN is trained online (data received sequentially), interference can occur between the representations that are learned [8]–[10]. Interference is loosely defined as when a newly learned representation affects the performance of the NN on previously seen data in a negative way (i.e. the old representation becomes worse due to the newly learned one). A potentially useful intuition for when interference can occur is when the representation for some input x_1 is dense (a large number of neurons are non-zero), since then, if another input x_2 is sufficiently different (uncorrelated) from x_1 , its representation should be quite different. However, for a different representation to be learned for x_2 , the NN must update the weights that contribute to the representation of x_1 , thus making the representation for x_1 worse to learn a better representation for x_2 .

A method that has been found to reduce interference, is to enforce sparse representations (a large number of neurons are approximately zero) [11]–[14]. We revisit the example above with inputs x_1 and x_2 . If the representation for x_1 is sparse, then the representation that is learned for x_2 can use weights that are unused for the representation of x_1 , thus causing no

interference between the representations for x_1 and x_2 . The best way to enforce sparsity is an active topic of research. A detailed discussion of related work can be found in Section I-A.

One way to enforce sparsity is by design, a method which does this is using a *fuzzy tiling activation* (FTA) [15,]. FTA is an activation function that can be used at each layer of a NN, similar to ReLU or tanh. Different from ReLU and tanh is that FTA has a vector output instead of a scalar output. Scalar quantities are the input whereas a one-hot vector becomes the corresponding output. The output of FTA can be thought of as mapping a scalar $z \in \mathbb{R}$ to one of k evenly spaced bins in $\mathbb{R}^k$. A critical step to using FTA effectively is setting the tiling bound parameter u . This is because z should be in $[-u, u]$ to work well. However, just satisfying this is usually not enough for good performance (as one could easily set u to be very large). For instance, if $u = 10$ and $k = 4$ then the bins would be $[-10, -5)$, $[-5, 0)$, $[0, 5)$, $[5, 10]$ and if z really takes values in $[0, 1]$ then it will always map to the same bin (bin 3 since $[0, 1] \subset [0, 5)$). Thus, as discussed by [15,], setting the tiling bound is important; however, no good solution exists for setting it other than performing a search. The sensitivity to the tiling bound was especially prominent in the work of [15,] when using FTA with a deep Q-network (DQN) [5,] in the LunarLander RL environment.

As such, in this work we aim to do two things:

- 1) We aim to reproduce the FTA vs ReLU using DQN in LunarLander experiments presented in [15,], to confirm that indeed FTA is sensitive to the tiling bound in this setting.
- 2) Next, we aim to test if the simple technique of normalizing the values (using tanh, Batch Norm, or Range Norm) passed to FTA can remove the need for tuning the tiling bound parameter in FTA.

Experiments related to the first point are shown in section III-A, indicating that the experiments were reproducible and nearly identical results were obtained as in [15,].

To answer the question in the second point we normalize the values passed into FTA using three methods: passing them into tanh, Batch Norm, or Range Norm (all to be defined in Section III-B). The method of using tanh to normalize the input values was discussed in [15], but not tested. [15]

mentioned that using this method might suffer from vanishing gradients. However, we did not see any indications of this in our experiments and obtained very positive results for tanh. We hypothesize that normalizing the values should solve the need to set tiling bounds using search, since normalizing will ensure that the values passed into FTA are in $[-1, 1]$, and thus the tiling bound can always be set to $[-1, 1]$. We found that tanh was most effective, while Batch Norm and Range Norm did not perform as well. Experiments highlighting these results are shown in Section III-B.

A. Related Work

Representation learning can influence learning efficiency, positively through generalization but negatively through interference [11], [16]–[18,]. Learned representations using neural networks are especially vulnerable to interference due to aggressive generalization, as updates to one feature’s estimates change the estimates of later features in unexpected ways, degrading model accuracy. Interference in features learned by neural networks is even worse when trained on temporally correlated data [19]–[21,]

Several authors [11], [12], [22], [23,] demonstrated how scattered representations tend to reduce the impact of interference for learning operations: few features stay active for each input, and thus each update is less likely to interfere with the weights of other features. Sparse feature spaces can leave an interpretable impact as outputs can be traced to the activation of the most input-discriminative features. However, learning such sparse representations in the online setting remains an open problem.

Existing strategies to ensure sparsity are to pre-train representations offline using regularizers [11,] or meta-learning [12], or online methods using regularizers with replay buffers [23]. While sparsity regularizers do improve learning, they often lead to high levels of dead neurons that degrade the learned representation over time [23,]. Kernel representations can also provide sparse representations online, but do not scale well to large problems due to their computational complexity [15,]. Therefore, there is motivation to find a simpler method to ensure sparsity in learned representations that is easy to implement, train, and scale to larger input spaces.

FTA is an approach to achieve sparsity by design, where mechanisms to activate can be deployed to the various layers found in a selected neural network. The mechanisms provides adjustable sparsity without the need for pre-training or adjustments to the loss function. The FTA function aggregates inputs of any feature layer into bins with differentiable, overlapping zones. Much like Tile Coding [24,], FTA allows learning to be discriminative between different input values through binning, but unlike Tile Coding, FTA maintains input generalization through learning activations within the differentiable regions between bins. When applied to DQN and Deep-Deterministic-Policy-Gradient, or DDPG for short, [25,], FTA repeatedly shows a likelihood of improving agent performance for discrete and continuous control environments, often outperforming activations like ReLU and tanh [15,].

Furthermore, FTA-applied networks can often perform better without target networks, circumventing the sample inefficiency problem inherent in their use [15,]. A recent study by [26] on transferring learned representations to networks performing auxiliary tasks found that FTA-based representations transferred better and more consistently than ReLU-based representations for less similar tasks.

II. PRELIMINARIES

Upon introducing FTA and the Rectified Linear Unit (ReLU) activation. Moreover, we formally define the RL setting and the Deep Q-learning (DQN) algorithm which we use for our experiments.

A. Fuzzy Tiling Activation

First, we introduce the simpler *tiling activation* (TA) and then extend it to the FTA. A TA is a function $\phi : \mathbb{R} \rightarrow \mathbb{R}^k$, which expects inputs $z \in [-u, u]$ and maps them to one-hot vectors (standard basis) in $\mathbb{R}^k$. The $[-u, u]$ is referred to as the tiling bound. For example, if $u = 10$ and $k = 4$, then any $z \in [-10, -5)$ would correspond to $(1, 0, 0, 0) \in \mathbb{R}^k$, any $z \in [-5, 0)$ would be linked to $(0, 1, 0, 0)$, any $z \in [0, 5)$ would be mapped to $(0, 0, 1, 0)$, and any $z \in [5, 10]$ would be mapped to $(0, 0, 0, 1)$. [15] show that the TA can be implemented efficiently as follows. Assume you want evenly spaced bins of size $\delta > 0$, and $k = 2u/\delta$, where WLOG u was chosen such that it is divisible by δ . Define the tiling vector

$$\mathbf{c} = (-u, -u + \delta, l + 2\delta, \dots, u - 2\delta, u - \delta) \in \mathbb{R}^k$$

Then the TA can be defined as

$$\phi(z) = \mathbb{1} - I_+(\max(\mathbf{c} - z\mathbb{1}, 0) + \max((z - \delta)\mathbb{1} - \mathbf{c}, 0))$$

where $\mathbb{1} \in \mathbb{R}^k$, and $I_+(\cdot)$ indicate returns equal to 1 for positive inputs and zero in other cases, and applied element-wise to vectors.

An issue with the TA is that it has zero derivatives almost everywhere. To solve this issue, [15] proposed modifying the TA for a fuzzy tiling activation (FTA). The FTA can be implemented as follows. Define a fuzzy version of the indicator function as

$$I_{\eta,+} = I_+(\eta - x)x + I_+(x - \eta)$$

where $\eta \geq 0$. Then the FTA can be defined as

$$\phi_\eta(z) = \mathbb{1} - I_{\eta,+}(\max(\mathbf{c} - z\mathbb{1}) + \max((z - \delta)\mathbb{1} - \mathbf{c}, 0)).$$

where $I_{\eta,+}$ is applied element-wise to vectors. The reason for the use of fuzzy in the name of FTA can be understood from the definition of $I_{\eta,+}$. The term $I_+(\eta - x)$ is 1 when $x < \eta$, while when $x > \eta$, $I_+(x - \eta)$ is 1. Thus, when $x < \eta$ we have that $I_{\eta,+}$ evaluates to x , giving a smoother (fuzzy) indicator function. Notice, that this means for $x < \eta$ the derivative is non-zero, allowing us to use backpropagation to train out networks. Also, when $\eta = 0$ the original indicator function I_+ can be recovered.

B. ReLU Activation

We will be comparing FTA to the ReLU activation later. Thus, we formally introduce it now. ReLU is defined as the positive part of its argument, formally:

$$\phi(z) = \max(0, z)$$

C. Reinforcement Learning

Markov Decision Process (MDP) accurately represents a reinforcement learning situation. The tuple $(\mathcal{S}, \mathcal{A}, \mathbb{P}, R, \gamma)$, where $\mathcal{S}$ is that state space, $\mathcal{A}$ is the action space, $\mathbb{P}$ is the transition probability kernel, $R : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow \mathbb{R}$ represents the reward, and $\gamma \in [0, 1]$ is the reduction. A policy $\pi : \mathcal{S} \rightarrow \mathcal{M}_1$ is a map from states to distributions over actions, where $\mathcal{M}_1$ is the set of all probability measures over actions $\mathcal{A}$. Then, an agent (policy) interacts with an MDP as follows. At each instance $t \in 1, 2, \dots$, a state is captured as defined by $s_t \in \mathcal{S}$. The agent responds per $a_t \sim \pi(s_t)$ before moving to a position described by $s_{t+1} \sim \mathbb{P}(s_t, a_t)$, for which the reward runs along $R(s_t, a_t, s_{t+1})$.

Given below is an action-value function with π policy.

$$Q_\pi(s, a) := \mathbf{E}[G_t | S_t = s, A_t = a; A_{t+1:\infty} \sim \pi]$$

where $G_t := \sum_{t=0}^{\infty} \gamma^t R(s_t, a_t, s_{t+1})$ is returned. We expect the agent to increase its reward for each state in $\mathcal{S}$.

D. Deep Q-Learning

When using Deep Q-Learning (DQN) [5,] in the experiments we performed, as explained in section II-C, the agent aims to interact with the ambient settings to enhance expected rewards. The ideal and most efficient action-value function is shown as

$$Q^*(s, a) = \max_{\pi} Q_\pi(s, a)$$

In Q-Learning the action-value function is iteratively updated by using the following (Bellman) equation to approach an action value function $Q_{t+1}(s, a) = \mathbf{E}[R(s_t, a_t, s_{t+1}) + \gamma \max_{a_{t+1} \in \mathcal{A}} Q_t(s_{t+1}, a_{t+1}) | S_t = s, A_t = a, S_{t+1} = s_{t+1}]$. This optimal function is $Q_t \rightarrow Q^*, t \rightarrow \infty$. A NN approximates with weights θ and approaches the action-value function, $Q(s, a; \theta)$. The agent's experience $(s_{t-1}, a_{t-1}, s_t, R(s_{t-1}, a_{t-1}, s_t))$ at each time step t is stored into a *replay memory*. A small sample $\mathcal{B}_t$ is selected randomly from the replay memory to calculate the loss function.

The NN training, which includes minimization of the given loss function at every interval t and then performing a gradient descent update

$$L_t(\theta_t) = \sum_{s, a, r, s' \in \mathcal{B}_t} [r + \gamma \max_{a'} Q(s', a'; \theta_{t-1}) - Q(s, a; \theta_t)]^2.$$

The agent's behavior policy is an ϵ -greedy mechanism, which randomly selects per probability ϵ and is characterized by greediness with respect to $Q(s_t, \cdot; \theta_t)$ otherwise.

III. EXPERIMENTS

The LunarLander environment, from OpenAI Gym [27] is a trajectory rocket optimization problem, where the goal is to land a moon lander on a landing pad. The state space of the environment $\mathcal{S}$ is compromised of 8 attributes; x and y coordinates, v_x and v_y velocities, θ and v_θ angular velocities, and 2 booleans to show if either leg is touching the ground. At each timestep t the action space

$\mathcal{A}(s_t) = \{\text{do nothing, fire left engine, fire main engine, fire right engine}\}.$

The reward for making it onto the landing pad earns 100 to 140 points, with an additional 100 points if it stops on it, hence a solved episode is +200 points. Firing the main engine has a -0.3 and the side engines -0.03 points penalty, and the lander gets -100 points if it crashes. If the agent crashes, leaves the viewport, or remains stationary without touching another body, the episode will terminate.

We use a student's t-distribution to display 95% confidence intervals for all of the plots in this section. The confidence intervals appear as the shaded regions in the plots. The validity of using such confidence intervals is investigated in Section III-C.

A. Reproducibility Experiment

Following the same LunarLander experiment settings as in [15], we use a two-layer neural network along a different activation function in the ultimate hidden layer as a primary difference. For DQN and DQN-Large, a ReLU activation mechanism applies to each layer save the last one, which is linear. Since DQN with FTA activation function expands the number of features, its last hidden layer is $k = (l - u)/\delta$ times bigger than the DQN, hence we use DQN-Large on the final layer that is of exactly the same size of DQN-FTA with and without the target network.

For DQN-FTA in the last layer we set $[l, u] = [-20, 20]$ and $\eta = \delta = 2.0$, hence $k = 40/2 = 20$.

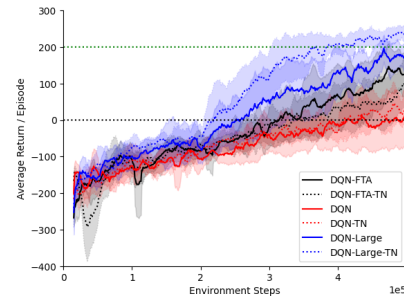


Fig. 1. Representations of **DQN-FTA(black)**, **DQN(red)** and **DQN-Large(blue)** in LunarLander. The **sprinkled** line shows the relevant network's algorithmic training. The results were taken as an average of 10 trials.

In figure 1, we observe that, while DQN-FTA with and without the target network performs slightly better than DQN,

DQN-Large outperforms both DQN and DQN-FTA. This is the same behavior that was observed by [15,] for this experiment, thus we were able to reproduce the experiment.

To show the sensitivity of DQN-FTA to the tiling bounds, we repeat the experiment for DQN-FTA only, with different values of $u \in \{0.1, 1, 10, 50, 100\}$, $l = -u$ with different number of tiles $k \in \{16, 24, 128\}$ for each tiling bounds.

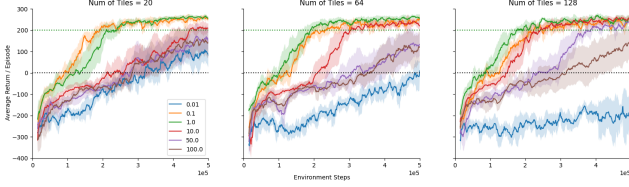


Fig. 2. Evaluation learning curves of DQN-FTA using different values of tiling bounds: $u \in \{0.1, 1, 10, 50, 100\}$, $l = -u$, with different number of tiles $k \in \{16, 64, 128\}$. Shown is the average result obtained from 10 trials.

Figure 2 depicts the results of the experiment. The DQN-FTA in LunarLander achieves its best when we set u to a reasonably small value (approximately $u = 1$). However, it performs poorly when it is set too large ($u = 10, 50, 100$) or small ($u = 0.01$). With $u \in \{10, 50, 100\}$, performance improves by increasing the number of tiles k , but it gets worse when u is very small. The improved performance when the number of tiles k is increased can be understood by looking at the case when $u = 10$ and $k = 128$. Notice that $u = 10$ is 10 times larger than $u = 1$ and $k = 128$ is approximately 10 times larger than $k = 16$. Thus, for the range $[-1, 1]$ there will be approximately the same number of bins when $u = 10, k = 128$ as when $u = 1, k = 16$. This explanation explains why the green line ($u = 1$) when $k = 16$ is approximately the same as the red line ($u = 10$) when $k = 128$.

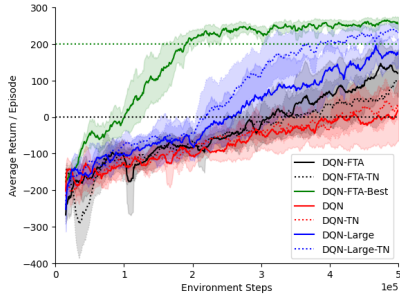


Fig. 3. Evaluation learning curves of **DQN-FTA(black)** ($u = 20, k = 40$), **DQN(red)**, **DQN-Large(blue)** and **DQN-FTA(green)** ($u = 1, k = 64$). The dotted line shows the algorithms trained with target network.

We found FTA sensitive to the tiling bound (u), hence only when choosing the right value of u we observed that it outperforms both DQN and DQN-Large. We found that the best value $u = 1$ for DQN-FTA without a target network was $u = 1$. Figure 3 shows a plot of the above-described DQN-FTA network compared to the untuned DQN-FTA ($u = 20$) and DQN using ReLU activation instead of FTA. This clearly indicates that when the value of u is set appropriately, DQN-FTA can outperform DQN with ReLU activation.

B. Normalizing Experiment

In our experiments, detailed in section III-A, it can be observed that the performance of FTA is quite sensitive to the tiling bound $[-u, u]$. This reinforces the observation made in [15]. In figure 2 we can see that the best performance on LunarLander was achieved with $u = 1$, and it got worse as we increased or decreased u . It can be inferred intuitively, that for a small value of u , many inputs to FTA may be out of its range providing zero gradients. Besides, for bigger values of u , multiple points of input can nurture similar tiles. In both scenarios, the results are numerous inactive neurons and enhanced interference. This means there is a specific value of u which would work best for any specific environment. Since it is not straightforward to observe the range of outputs from a Neural Network layer, u can not be set mathematically. It has to be tuned manually by sweeping over a range of values in multiple experiments, which increases the cost of a project.

We hypothesize that if the inputs to FTA were scaled to a certain range, the performance would be less sensitive to the tiling bounds. We try three different methodologies to achieve this. First, using a Batch Normalization [28,] layer before FTA. Second, using a tanh activation layer before FTA. And finally, using a Range Normalization layer before FTA.

Batch Normalization scales its input x to a learned mean β and variance γ . It is defined as

$$y = \frac{x - E[x]}{\sqrt{\text{Var}[x] + \epsilon}} * \gamma + \beta \quad (1)$$

By using a Batch Norm layer before FTA, we expect it to learn the best γ and β for any particular tiling bound of FTA.

On the other hand, tanh is an activation function that maps the input to a continuous range of values between -1 and 1. For bigger inputs, the output tends closer to 1, and in cases of smaller input instances, the output points gravitate to -1. This range of outputs can be controlled by multiplying them through non-vector functions as demonstrated below.

$$y = \frac{e^x - e^{-x}}{e^x + e^{-x}} \quad (2)$$

The motivation behind using tanh is to control the range of inputs to FTA so that a standard tiling bound of $u = -l = 1$ would work in every case, regardless of the agent or the environment.

Range Normalization is used to scale the values to fall within a specific range. We use Range Normalization to scale the inputs to FTA, to the range of -1 and 1. It is defined as

$$y = \frac{x - x_{\min}}{x_{\max} - x_{\min}} * 2 - 1 \quad (3)$$

Similar to tanh, the reason for using Range Normalization is to ensure the range of inputs to FTA is between -1 and 1.

To test this hypothesis, we run experiments with tanh, Batch Norm, and Range Norm as the layer preceding FTA on Lunar Lander. One, with the best performing FTA tiling bound ($u = 1$) and two, with the worst performing FTA tiling bounds

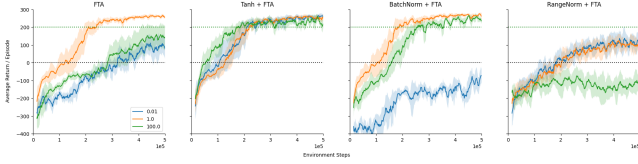


Fig. 4. Note the curves plotting periodic results against environment intervals for Batch Norm, Range Norm and tanh as layers preceding FTA, compared to DQN-FTA without a normalization layer preceding FTA.

($u = 0.01, 100$). All other parameters and configurations are kept the same as those in section III-A.

Figure 4 depicts the results of these experiments in LunarLander, compared to DQN-FTA with the best FTA tiling bounds and the two worst-performing tiling bounds. When tanh is used as the preceding layer to FTA to normalize its inputs, we observe good performance with all the tested tiling bounds and a significant improvement over DQN-FTA with the worst tiling bounds ($u = 0.01, 100$). Range Normalization performs poorly across all the tiling bounds. While Batch Normalization improves the performance of DQN-FTA with the highest tiling bound, it performs very poorly when the tiling bound is very small. Overall, it is clear that *tanh* normalization is effective regardless of the tiling bound in the LunarLander environment.

1) *Cartpole*: To test the effectiveness of the tanh normalization technique across environments we also performed experiments on the CartPole environment from OpenAI Gym [27]. The CartPole environment starts as a pole balanced atop a cart, with the objective being to maintain the pole’s stability even when the cart shifts to the left or to the right. The state space of the CartPole environment contains four values: cart stance, cart velocity, pole angle, and pole angular velocity. The directions involve moving the cart to the left, and then moving it to the right. With the objective being to ensure the pole does not fall, a +1 reward ensues for each step, and the episode terminates when the pole is at an angle greater than $\pm 12^\circ$.

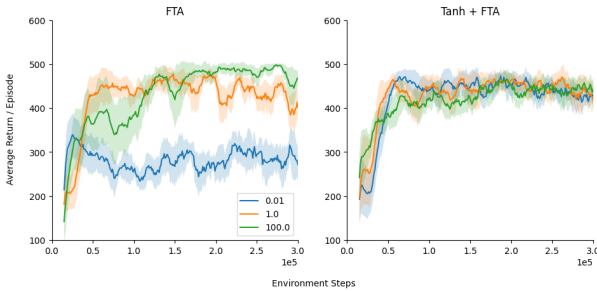


Fig. 5. Each agent learning curve is generated using a u and all are trained without target networks.

We first tested to see how DQN-FTA without a target network performs with three different tiling bounds ($u = 0.01, 1, 100$) and $k = 20$. The results can be seen on the left plot in Figure 5. Unlike in LunarLander, a tiling bound of $u = 100$ performs best, while $u = 1$ performs worse. Importantly, this highlights the sensitivity of FTA to the tiling bound parameter in the

CartPole environment as well as LunarLander.

We then applied tanh normalization before the FTA layer and found this improved the performance of $u = 0.01, 1$ to be similar to that of $u = 100$. The right plot in Figure 5 illustrates that DQN-FTA with tanh normalization at all tiling bounds perform similarly well to the best tiling bounds for DQN-FTA without tanh normalization, for all the tiling bounds we tested ($u = 0.01, 1, 100$). Thus, the experimental results in the CartPole environment further confirm that using tanh normalization before an FTA layer can potentially remove the need for tuning the tiling bound u .

C. Distribution of Returns

We observed that DQN-FTA had noticeably different returns at 500k steps between seeds, wherein some seeds performed significantly better than others. To better understand agent performance with respect to initialization stochasticity, we investigate the distribution of mean episodic returns for ReLU, FTA, and FTA-tanh over various time steps. Additionally, student’s t confidence intervals and averaging over runs in evaluation learning curves assume that mean episodic returns are uni-modal and normally distributed; Violating these assumptions would misrepresent the robustness of the agent and the average return to which it converges. A bi-modal distribution of average episodic returns at the final stage of learning (ie: 500k steps) would indicate that an agent is sensitive to weight initialization or environmental stochasticity and therefore demonstrate inconsistent learning and unreliable behavior.

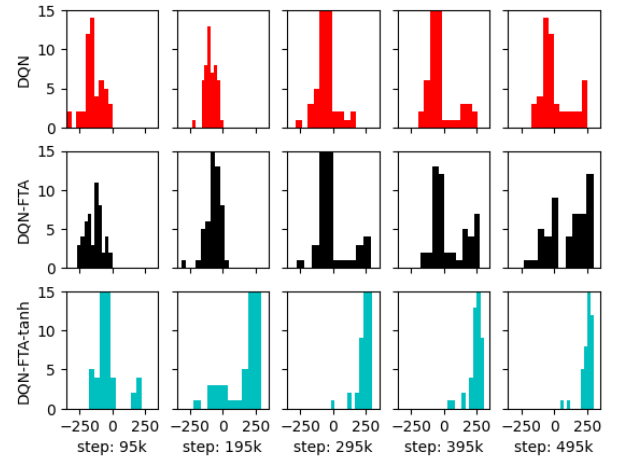


Fig. 6. Histogram of mean episodic returns for **DQN**(red), **DQN-FTA**(black), and **DQN-FTA-tanh**(cyan) over time step learning progression in LunarLander. There are 50 runs per agent, with the mean episodic return values at each time step divided into 10 bins between the range $[-350, 350]$. All three agents do not use target networks.

Figure 6 shows the distribution of mean episodic returns for DQN, (untuned) DQN-FTA, and DQN-FTA-tanh. DQN and DQN-FTA has the same parameters as in III-A, and DQN-FTA-tanh has the same parameters as in III-B. The FTA agents have the same number of bins, $k = 20$, with DQN-FTA-tanh differing from DQN-FTA by a tanh activation

layer before the FTA layer. We can observe some general distribution characteristics and trends; For DQN and DQN-FTA, distributions of returns at the beginning of learning appear uni-modal and shift away from the mean of their distributions near the end of learning, while the distribution of returns for DQN-FTA-tanh remain uni-modal and appear to converge towards the mean of its distribution. At 495k time steps, DQN and DQN-FTA distributions appear bi-modal while the DQN-FTA-tanh distribution appears strongly uni-modal and more normally distributed. While 50-run sample sizes per agent is relatively small hypothesis testing, our preliminary results in LunarLander show that 1) DQN and DQN-FTA distributions of returns may violate the normality and uni-modality assumptions near the end of training, an important observation on performance stability under stochasticity in the environment and weight initializations, and 2) DQN-FTA-tanh appears to maintain those assumptions throughout learning, illustrating that tanh normalization can improve FTA stability.

IV. DISCUSSION

After exploring FTA through empirical experimentation as an extension to the original FTA paper by [15,]. We first reproduced the learning curve evaluation for DQN-FTA, DQN, and DQN-Large in LunarLander, and found similar trends. When comparing the untuned FTA agent ($u = 20$) against the tuned ReLU agents, we found that FTA and DQN-Large reached higher average returns by 5×10^5 time steps than [15]. It is currently unknown why our FTA and DQN-Large implementations perform better than the original paper. There are many potential reasons: stochasticity in our runs and initialization, our specific seeds used to evaluate performance, or even the implementation framework (we used PyTorch while [15] used TensorFlow) could all influence the empirical outcome of experiments. While we only average over 10 runs compared to the 20 performed in the original paper, this should not have contributed to the performance discrepancy given our relatively tight confidence intervals between runs.

Further investigation into the plausible causes may be useful; however, we believe that since the same trends between networks were observed it is sufficient to claim we were able to reproduce the LunarLander experiments from [15,].

Our second experiment reproduces the bound sensitivity experiment comparing FTA with different tiling bounds and across different numbers of tiles. Our results share the conclusion that moderately small values for u ($u \in 0.1, 1$) perform better than the extremes ($u \in 0.01, 10, 50, 100$) [15,]. We also found that increasing the number of tiles improves the performance of larger tiling bounds ($u \in 50, 100$) while an extremely small u gets worse ($u = 0.01$). This is expected, as increasing the number of tiles given a large tiling bound reduces δ , thereby reducing interference between bins. In addition, increasing the number of tiles given an extremely small tiling bound greatly increases δ and results in high interference between bins.

While [15] addressed the *out-of-boundary* and *gradient vanishing* problem when $z \notin [l, u]$, we address the over-generalization problem due to incorrect bounds wherein the

majority of $z \in [l, u]$ aggregate into a small range of the available bins. For example, if the inputs to FTA are $z = x^\top w \in [-1, 1]$ while the range $[l, u] = [-20, 20]$ and $\delta = \eta = 2$ such that number of bins $k = 20$ and $c = (l, l + \delta, l + 2\delta, \dots, u - 2\delta, u - \delta, u) = (-20, -18, -16, \dots, 16, 18, 20)$ (like the LunarLander experiment in section III-A), then $FTA(z)$ will only bin z into two bins, between the ranges $[-2, 0)$ and $[0, 2)$. When the $[l, u]$ bounds extend beyond the standard input to activation range, the sparsity provided by FTA is limited due to the number of unused bins. To address the bound selection problem, we perform normalizing experiments using Range Normalization, tanh activation, and Batch Normalization. We found that tanh normalization is bound-agnostic and improves the performance of FTA without requiring additional tuning for the environment. In section III-B, we show that tanh-normalized FTA performs the same regardless of the tiling bound $[l, u]$ given a fixed number of bins k . When comparing against the agents in the LunarLander experiments, tanh-normalized DQN-FTA performs as well as a tuned FTA and better than all the other agents near the end of learning. tanh-normalized DQN-FTA also performs better than untuned FTA in Cartpole, extending the robustness tanh normalization within other environments where the input range for $z \in [l, u]$ is unknown and agents are untuned.

Lastly, we evaluate the distribution of mean episodic returns for DQN, (untuned) DQN-FTA, and DQN-FTA with tanh normalization in LunarLander. We find that near the end of learning, DQN and DQN-FTA have more bi-modal distributions whereas DQN-FTA with tanh normalization is more uni-modal. This potentially demonstrates that tanh-normalized FTA improves agent stability to randomness in the initialization of network weights and environment settings compared to agents using ReLU or untuned FTA. Untuned DQN-FTA's bi-modally distributed returns could also potentially explain the poor performance in LunarLander reported by [15]. Although our distribution experiments are preliminary, and that hypothesis testing with significantly more sample runs is required to more conclusively address distribution normality and modality, figure 6 illustrates how inconsistent the returns can be for an untuned FTA agent. Furthermore, our results emphasize the sensitivity of FTA tiling bounds $[l, u]$ and the importance of a bound-agnostic solution to FTA for its reliability and widespread adoption.

Comparing against the agents in the LunarLander experiments of section III-A, tanh-normalized DQN-FTA performs as well as a tuned FTA and better than all the other agents near the end of learning. The experiments performed in the CartPole environment indicate a similar conclusion. Evaluating the robustness of tanh-normalized DQN-FTA across more environments is left as useful future work.

V. CONCLUSION

To summarize, our study aimed to investigate the properties of FTA through empirical experimentation. We observed that neural networks perform well on challenging problems, but interference can be reduced by enforcing sparse representations.

One method to achieve this is through fuzzy tiling activation (FTA). Our research found that using tanh normalization in FTA improved its performance without the need for additional tuning, and it was found to be bound-agnostic. In conclusion, our study suggests that FTA with tanh normalization has the potential to enhance the robustness of neural networks and may be beneficial in various environments. Further research could explore its efficacy in more diverse settings.

Our research topic focused on investigating the properties of fuzzy tiling activation (FTA) through empirical experimentation. Neural networks have shown impressive performance on various challenging problems. However, one significant challenge in neural networks is the issue of interference, which can occur when multiple inputs activate the same neurons. Enforcing sparse representations has been proposed as a solution to address this issue.

One method to enforce sparsity in neural networks is through the use of FTA, to separate input field as sets of fuzzy tiles. Each tile is associated with a neuron that is only activated when the input falls within its corresponding fuzzy region. This helps to enforce a sparse representation of the input space and can reduce interference.

In our research, we investigated the impact of using tanh normalization in FTA to improve its performance. We found that using tanh normalization was bound-agnostic and improved the performance of FTA without requiring additional tuning for the environment. This is because tanh normalization helped to keep the activations of the neurons within a reasonable range, which improved the stability and convergence of the network during training.

REFERENCES

- [1] A. H. Ismail, T. A. Soliman, M. Rihan, and M. I. Dessouky, "Deep learning-based beamforming for millimeter-wave systems using parametric relu activation function," *Wireless Personal Communications*, pp. 1–12, 2023.
- [2] S. Wei and E. Lau, "Variational bayesian neural networks via resolution of singularities," *arXiv preprint arXiv:2302.06035*, 2023.
- [3] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, "Language models are few-shot learners," *Advances in neural information processing systems*, vol. 33, pp. 1877–1901, 2020.
- [4] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," *Communications of the ACM*, vol. 60, no. 6, pp. 84–90, 2017.
- [5] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. Riedmiller, "Playing atari with deep reinforcement learning," *arXiv preprint arXiv:1312.5602*, 2013.
- [6] D. Silver, J. Schrittwieser, K. Simonyan, I. Antonoglou, A. Huang, A. Guez, T. Hubert, L. Baker, M. Lai, A. Bolton *et al.*, "Mastering the game of go without human knowledge," *nature*, vol. 550, no. 7676, pp. 354–359, 2017.
- [7] M. J. Tilley, M. Miller, and D. Freedman, "Artificial neuronal ensembles with learned context dependent gating," *arXiv preprint arXiv:2301.07187*, 2023.
- [8] Y. Chandak, G. Theodorou, J. Kostas, S. Jordan, and P. Thomas, "Learning action representations for reinforcement learning," pp. 941–950, 2019.
- [9] H. Caselles-Dupré, M. Garcia-Ortiz, and D. Filliat, "Continual state representation learning for reinforcement learning using generative replay," *arXiv preprint arXiv:1810.03880*, 2018.
- [10] S. Madjheurem and L. Toni, "Representation learning on graphs: A reinforcement learning application," pp. 3391–3399, 2019.
- [11] V. Liu, R. Kumaraswamy, L. Le, and M. White, "The utility of sparse representations for control in reinforcement learning," vol. 33, pp. 4384–4391, 2019.
- [12] K. Javed and M. White, "Meta-learning representations for continual learning," *Advances in Neural Information Processing Systems*, vol. 32, 2019.
- [13] J. Rafati and D. C. Noelle, "Learning sparse representations in reinforcement learning," *arXiv preprint arXiv:1909.01575*, 2019.
- [14] S. Gai, "Theory of reduced biquaternion sparse representation and its applications," *Expert Systems with Applications*, vol. 213, p. 119245, 2023.
- [15] Y. Pan, K. Banman, and M. White, "Fuzzy tiling activations: A simple approach to learning sparse representations online," *arXiv preprint arXiv:1911.08068*, 2019.
- [16] J. A. Bullinaria, "Representation, learning, generalization and damage in neural network models of reading aloud," *Submitted to Psychological Review*, 1994.
- [17] S.-A. Rebuffi, A. Kolesnikov, G. Sperl, and C. H. Lampert, "icarl: Incremental classifier and representation learning," pp. 2001–2010, 2017.
- [18] L. Le, R. Kumaraswamy, and M. White, "Learning sparse representations in reinforcement learning with sparse coding," pp. 2067–2073, 2017.
- [19] V. Liu, A. White, H. Yao, and M. White, "Towards a practical measure of interference for reinforcement learning," *arXiv preprint arXiv:2007.03807*, 2020.
- [20] E. Bengio, J. Pineau, and D. Precup, "Interference and generalization in temporal difference learning," vol. 119, pp. 767–777, 13–18 Jul 2020.
- [21] T. Zhang, X. Wang, B. Liang, and B. Yuan, "Catastrophic interference in reinforcement learning: A solution based on context division and knowledge distillation," *IEEE Transactions on Neural Networks and Learning Systems*, pp. 1–15, 2022.
- [22] S. Ghiassian, B. Rafiee, Y. L. Lo, and A. White, "Improving performance in reinforcement learning by breaking generalization in neural networks," *arXiv preprint arXiv:2003.07417*, 2020.
- [23] J. F. Hernandez-Garcia and R. S. Sutton, "Learning sparse representations incrementally in deep reinforcement learning," *ArXiv*, vol. abs/1912.04002, 2019.
- [24] A. A. Sherstov and P. Stone, "Function approximation via tile coding: Automating parameter choice," pp. 194–205, 2005.
- [25] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, "Continuous control with deep reinforcement learning," *arXiv preprint arXiv:1509.02971*, 2015.
- [26] H. Wang, E. Miah, M. White, M. C. Machado, Z. Abbas, R. Kumaraswamy, V. Liu, and A. White, "Investigating the properties of neural network representations in reinforcement learning," 2022.
- [27] G. Brockman, V. Cheung, L. Pettersson, J. Schneider, J. Schulman, J. Tang, and W. Zaremba, "Openai gym," *arXiv preprint arXiv:1606.01540*, 2016.
- [28] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," pp. 448–456, 2015.